# SEEKRIGHT Ingestion Pipeline: Final Implementation Report

## Project Overview

This report summarizes the finalization of Phases 2-5 of the SEEKRIGHT Ingestion Roadmap. The pipeline is designed to handle YouTube lecture ingestion, transcription (via Whisper), deterministic chunking, and provides a stable interface for downstream RAG (Retrieval-Augmented Generation) work.

## Project Architecture & File Tree

```
backend/
├── app/
│   ├── main.py                # API entry point & automatic table initialization
│   ├── models.py              # SQLAlchemy models (SESSIONS, TRANSCRIPTS, etc.)
│   ├── database.py            # SQLite configuration with WAL & Busy Timeout
│   ├── schemas.py             # Pydantic schemas for API I/O
│   ├── routers/
│   │   ├── auth.py            # Authentication endpoints
│   │   ├── session.py         # Session management endpoints
│   │   └── query.py           # Query/RAG endpoints
│   └── services/
│       ├── session_service.py     # Core pipeline orchestration & lifecycle
│       ├── transcription_service.py # Audio download (yt-dlp) & Transcription (Whisper)
│       ├── chunk_service.py       # Deterministic text segmentation
│       └── embedding_service.py   # Dummy embedding contract (384-dim)
├── verify_ingestion_v2.py      # Comprehensive E2E validation suite
├── server_log.txt              # Captured server runtime logs
└── seekright.db                # SQLite Persistent Storage
```

## Technical Resolutions

### 1. Robust Dependency Management

- **FFmpeg Guard**: Implemented pre-transcription checks to detect FFmpeg. The system fails gracefully with an actionable `failure_reason` if the dependency is missing, preventing background crashes.
- **Whisper Integration**: Model loading is handled at the module level for efficiency, with support for automatic CPU/GPU scaling.

### 2. Data Integrity & Atomicity

- **Atomic Transactions**: Transcript persistence and chunk generation are wrapped in a single database transaction. This ensures that the system never enters a "partial chunking" state.
- **Deterministic Chunking**: Chunks are generated with fixed indexing and precise timestamps, ensuring retrieval accuracy for Member 4's FAISS implementation.

### 3. High Concurrency (SQLite Optimization)

- **WAL Mode**: Enabled Write-Ahead Logging to allow multiple concurrent readers and writers.

- **Busy Timeout**: Increased to 5000ms to handle momentary locks during heavy transcription/chunking operations.

### 4. Lifecycle & Observability

- **Duration Tracking**: Precise timing of the entire pipeline (from PENDING to COMPLETED/FAILED) is stored in the `SESSIONS` table.
- **Detailed Reporting**: `failure_reason` is captured at every potential failure point and exposed via the API status endpoint.

## Verification Results

The pipeline was validated using `verify_ingestion_v2.py` with the following results:

| Test Scenario | Status | Result |
| --- | --- | --- |
| **Pipeline Flow** | ✅ SUCCESS | Full lifecycle (PENDING → PROCESSING → ... → COMPLETED) validated. |
| **Invalid Input** | ✅ SUCCESS | Graceful 400 rejection and FAILED state handling. |
| **Concurrency** | ✅ SUCCESS | Simultaneous sessions processed without SQLite blocking. |
| **Environment Check** | ✅ SUCCESS | Detected missing FFmpeg and reported correctly without crashing. |

---

*Report generated by Antigravity on 2026-02-27.*